

Specifying and Verifying Future Conditions

Yahui Song, Darius Foo, Wei-Ngan Chin

October @ SAS 2025, Singapore



The existing solution

```
void *malloc (size_t size);  
// pre: size>0 ∧ _*  
// post: (ret=null ∧ ε) ∨ (ret≠null ∧ malloc(ret))  
// future: ret≠null →  $\mathcal{F}$  (free(ret))
```

$$\frac{f(\bar{x}) [\Phi_{pre}, \Phi_{post}, \Phi_{future}] \in \mathcal{E} \quad [\text{FV-Call}] \quad \Phi <: \Phi_{pre} \quad \{\Phi \circ \Phi_{post}\} e \{\Phi_e\} \quad \Phi_e <: \Phi_{future}}{\{\Phi\} f(\bar{x}); e \{\Phi_{post} \circ \Phi_e\}}$$

Three main limitations:

- ❑ Inefficient ($O(n^2)$) entailment checking
- ❑ Handle loops via unrolling
- ❑ Bug-finding (no incorrectly flagged safe code) over soundness (no missed violations)

Inefficient ($O(n^2)$) entailment checking

A use-after-free bug recorded from CWE-416

```
1 int main(int argc, char **argv) {  
2     char *buf1, *buf2, *buf3;  
3     buf1 = malloc(1);  
4     buf2 = malloc(1);  
5     free(buf2);  
6     buf3 = malloc(1);  
7     strncpy(buf2, argv[1], 1);    Use-after-free!  
8     free(buf1); free(buf3); }
```

Inefficient ($O(n^2)$) entailment checking

```
void *malloc (size_t size);  
// pre: size > 0 ∧  $\_*$   
// post: (ret = null ∧  $\epsilon$ ) ∨ (ret ≠ null ∧ malloc(ret))  
// future: ret ≠ null →  $\mathcal{F}$  (free(ret))
```

```
void free (void *ptr);  
// post: true ∧ free(ptr)  
// future: true ∧  $\mathcal{G}$  (!_(ptr))
```

```
char *strncpy(char *dest, const char *source, size_t num);  
// post: true ∧ strncpy(dest)
```

$f(\bar{x}) [\Phi_{pre}, \Phi_{post}, \Phi_{future}] \in \mathcal{E}$ [FV-Call]

$$\frac{\Phi <: \Phi_{pre} \quad \{\Phi \circ \Phi_{post}\} e \{\Phi_e\} \quad \Phi_e <: \Phi_{future}}{\{\Phi\} f(\bar{x}); e \{\Phi_{post} \circ \Phi_e\}}$$

```
1 int main(int argc, char **argv) {  
2     char *buf1, *buf2, *buf3;  
3     buf1 = malloc(1);  
4     buf2 = malloc(1);  
5     free(buf2);  
6     buf3 = malloc(1);  
7     strncpy(buf2, argv[1], 1);    Use-after-free!  
8     free(buf1); free(buf3); }
```

Inefficient ($O(n^2)$) entailment checking

```
void *malloc (size_t size);
// pre: size>0  $\wedge$   $\_*$ 
// post: (ret=null  $\wedge$   $\epsilon$ )  $\vee$  (ret!=null  $\wedge$  malloc(ret))
// future: ret!=null  $\rightarrow \mathcal{F}$  (free(ret))
```

```
void free (void *ptr);
// post: true  $\wedge$  free(ptr)
// future: true  $\wedge \mathcal{G}$  (!_(ptr))
```

```
char *strncpy(char *dest, const char *source, size_t num);
// post: true  $\wedge$  strncpy(dest)
```

$f(\bar{x}) [\Phi_{pre}, \Phi_{post}, \Phi_{future}] \in \mathcal{E}$ [FV-Call]

$$\frac{\Phi <: \Phi_{pre} \quad \{\Phi \circ \Phi_{post}\} e \{\Phi_e\} \quad \Phi_e <: \Phi_{future}}{\{\Phi\} f(\bar{x}); e \{\Phi_{post} \circ \Phi_e\}}$$

```
1 int main(int argc, char **argv) {
2   char *buf1, *buf2, *buf3;
3   buf1 = malloc(1);  $\leftarrow$  malloc(buf2).free(buf2).malloc(buf3).strncpy(buf2).free(buf1).free(buf3)  $\sqsubseteq$  F(free(buf1))
4   buf2 = malloc(1);  $\leftarrow$  free(buf2).malloc(buf3).strncpy(buf2).free(buf1).free(buf3)  $\sqsubseteq$  F(free(buf2))
5   free(buf2);  $\leftarrow$  malloc(buf3).strncpy(buf2).free(buf1).free(buf3)  $\not\sqsubseteq$  G(!_ (buf2))
6   buf3 = malloc(1);  $\leftarrow$  strncpy(buf2).free(buf1).free(buf3)  $\sqsubseteq$  F(free(buf3))
7   strncpy(buf2, argv[1], 1); Use-after-free!
8   free(buf1); free(buf3); }  $\leftarrow$  free(buf3)  $\sqsubseteq$  G(!_ (buf1))
                                     empty  $\sqsubseteq$  G(!_ (buf3))
```

A new solution for reasoning FCs

2. `char *buf1, *buf2, *buf3;`

3. `buf1 = malloc(1);`

4. `buf2 = malloc(1);`

5. `free(buf2);`

6. `buf3 = malloc(1);`

7. `strncpy(buf2,argv[1],1);`

A new solution for reasoning FCs

```
2. char *buf1, *buf2, *buf3;
{ (∃buf1, buf2, buf3. true ; ε ; _*) }
3. buf1 = malloc(1);
{ (∃buf1, buf2, buf3. buf1 ≠ null ; malloc(buf1) ; ℱ(free(buf1))) }
4. buf2 = malloc(1);
{ (∃buf1, buf2, buf3. buf1 ≠ null ∧ buf2 ≠ null ; malloc(buf1) · malloc(buf2) ;
  ℱ(free(buf1)) ∧ ℱ(free(buf2))) }
5. free(buf2);
{ (∃buf1, buf2, buf3. buf1 ≠ null ∧ buf2 ≠ null ; malloc(buf1) · malloc(buf2) · free(buf2) ;
  ℱ(free(buf1)) ∧ _* ∧ ℒ(!_ (buf2))) }
6. buf3 = malloc(1);
{ (∃buf1, buf2, buf3. buf1 ≠ null ∧ buf2 ≠ null ∧ buf3 ≠ null ; malloc(buf1) · malloc(buf2)
  · free(buf2) · malloc(buf3) ; ℱ(free(buf1)) ∧ ℒ(!_ (buf2)) ∧ ℱ(free(buf3))) }
7. strncpy(buf2, argv[1], 1);
{ (∃buf1, buf2, buf3. buf1 ≠ null ∧ buf2 ≠ null ∧ buf3 ≠ null ; malloc(buf1) · malloc(buf2)
  · free(buf2) · malloc(buf3) · strncpy(buf2) ; ℱ(free(buf1)) ∧ ⊥ ∧ ℱ(free(buf3))) ⇐ X }
FC Violation Found: subtracting “strncpy(buf2)” from “ℒ(!_ (buf2))” leads to false!
```

- ❖ Linear trace processing
- ❖ Embed FCs into program states
- ❖ Trace conjunction + subtraction

The existing solution

<pre>void *malloc (size_t size); // pre: size>0 ∧ _[★] // post: (ret=null ∧ ε) ∨ (ret≠null ∧ malloc(ret)) // future: ret≠null → $\mathcal{F}$ (free(ret))</pre>	$\frac{f(\bar{x}) [\Phi_{pre}, \Phi_{post}, \Phi_{future}] \in \mathcal{E} \quad [\text{FV-Call}] \quad \Phi <: \Phi_{pre} \quad \{\Phi \circ \Phi_{post}\} e \{\Phi_e\} \quad \Phi_e <: \Phi_{future}}{\{\Phi\} f(\bar{x}); e \{\Phi_{post} \circ \Phi_e\}}$
--	---

Three main limitations:

- ✓ Inefficient entailment checking **Embed FCs into the states + Trace subtraction**
- ❑ Handle loops via unrolling
- ❑ Bug-finding (no incorrectly flagged safe code) over soundness (no missed violations)

Predicates for Bags of Traces and Future Conditions

A false negative example from ProveNFix

```
1 void* mallocN(int n, void **arr,){
2     int i = 0;
3     while (i < n) {
4         arr[i] = malloc(4); i = i+1;}
5     return *arr;}
6
7 void main () {
8     void *arr[5]; mallocN (5, arr);
9     free(arr[0]);/* memory leak */}
```

Predicates for Bags of Traces and Future Conditions

```

1 void* mallocN(int n, void **arr,){
2   int i = 0;
3   while (i < n) {
4     arr[i] = malloc(4); i = i+1;}
5   return *arr;}
6
7 void main () {
8   void *arr[5]; mallocN (5, arr);
9   free(arr[0]);/* memory leak */}

```



$mallocN(n, arr) \equiv \text{req: } length(arr) \geq n$

$\text{ens: } (\exists i. \text{true} ; \text{pred}_t([0..n), i) ; \text{pred}_f([0..n), i))$

$\text{pred}_t(B, i) \equiv \Lambda_i^B (arr[i] \neq \text{null} \wedge \text{malloc}(arr[i])) \vee (arr[i] = \text{null} \wedge \epsilon)$

$\text{pred}_f(B, i) \equiv \Lambda_i^B (arr[i] \neq \text{null} \wedge \mathcal{F}(\text{free}(arr[i])))$

When reasoning about main():

```

8. void *arr[5]; mallocN (5, arr);
   { (∃ arr, i. length(arr) = 5 ; predt([0..5), i) ; predf([0..5), i) ) }

9. free(arr[0]);
   { (∃ arr, i. length(arr) = 5 ; predt([0..5), i) · free(arr[0]) ; predf([1..5), i) ∧ G(!_ (arr[0])) ) }

```

FC Violation Found: empty trace “ ϵ ” does not satisfy the obligation “ $\text{pred}_f([1..5), arr)$ ”!

(Specification) $[\text{req: } \pi \text{ ens: } \Delta]$
 (Post Summary) $\Delta ::= \bigvee (\pi ; \theta ; F)$

Predicates for Bags of Traces and Future Conditions

```

1 void* mallocN(int n, void **arr,){
2   int i = 0;
3   while (i < n) {
4     arr[i] = malloc(4); i = i+1;}
5   return *arr;}
6
7 void main () {
8   void *arr[5]; mallocN (5, arr);
9   free(arr[0]);/* memory leak */}

```

When reasoning about mallocN():

$$\frac{[\text{FV-While}] \quad \{(\pi \wedge \pi_g; \theta; F)\} e \{(\pi; \theta; F)\}}{\{(\pi; \theta; F)\} \text{ while } \pi_g \text{ do } e \{(\pi \wedge \neg \pi_g; \theta; F)\}}$$



$\text{mallocN}(n, \text{arr}) \equiv \text{req: } \text{length}(\text{arr}) \geq n$

$\text{ens: } (\exists i. \text{true}; \text{pred}_t([0..n], i); \text{pred}_f([0..n], i))$

$\text{pred}_t(B, i) \equiv \Lambda_i^B (\text{arr}[i] \neq \text{null} \wedge \text{malloc}(\text{arr}[i])) \vee (\text{arr}[i] = \text{null} \wedge \epsilon)$

$\text{pred}_f(B, i) \equiv \Lambda_i^B (\text{arr}[i] \neq \text{null} \wedge \mathcal{F}(\text{free}(\text{arr}[i])))$

```

3. while (i < n){
  {(\exists i. \text{true}; \text{pred}_t([0..i], i); \text{pred}_f([0..i], i))}
  4.   arr[i] = malloc(4);
  {(\exists i. \text{true}; \text{pred}_t([0..i+1], i); \text{pred}_f([0..i+1], i))}
  5.   i = i+1;
  {(\exists i. \text{true}; \text{pred}_t([0..i+1], i+1); \text{pred}_f([0..i+1], i+1))}
  6. } {(\exists i. i = n; \text{pred}_t([0..i], i); \text{pred}_f([0..i], i))} \rightsquigarrow
  {(\exists i. \text{true}; \text{pred}_t([0..n], i); \text{pred}_f([0..n], i))}

```

The existing solution

<pre>void *malloc (size_t size); // pre: size>0 ∧ _[★] // post: (ret=null ∧ ε) ∨ (ret≠null ∧ malloc(ret)) // future: ret≠null → $\mathcal{F}$ (free(ret))</pre>	$\frac{f(\bar{x}) [\Phi_{pre}, \Phi_{post}, \Phi_{future}] \in \mathcal{E} \quad [\text{FV-Call}] \quad \Phi <: \Phi_{pre} \quad \{\Phi \circ \Phi_{post}\} e \{\Phi_e\} \quad \Phi_e <: \Phi_{future}}{\{\Phi\} f(\bar{x}); e \{\Phi_{post} \circ \Phi_e\}}$
--	---

Three main limitations:

- ✓ Inefficient entailment checking **Embed FCs into the states + Trace subtraction**
- ✓ Handle loops via unrolling **Predicates for bags of traces and FCs**
- ❑ Bug-finding (no incorrectly flagged safe code) over soundness (no missed violations)

Soundness Formalization

- An instrumented semantics for the target language: $[s, \rho, F, e] \xrightarrow{\text{stack, execution trace}} [s', \rho', F', v]$
- Semantic model of trace specifications: $s, \rho \models \pi \wedge \theta$
- A set of forward verification rules: $\{(P; \theta_1; F_1)\} e \{(Q; \theta_2; F_2)\}$

```
Theorem soundness : forall P e Q t1 t2 rho1 rho2 s1 v s2 f1 f2 f3,  
  forward P t1 f1 e Q t2 f2 ->  
  P s1 ->  
  trace_model rho1 t1 ->  
  bigstep s1 rho1 f1 e s2 rho2 f3 v ->  
  Q v s2 /\ trace_model rho2 t2 /\ futureCondEntail f2 f3.
```

It only sound to strengthen the future conditions, so that we do not miss any violations.

The existing solution

<pre>void *malloc (size_t size); // pre: size>0 ∧ _[★] // post: (ret=null ∧ ε) ∨ (ret≠null ∧ malloc(ret)) // future: ret≠null → $\mathcal{F}$ (free(ret))</pre>	$\frac{f(\bar{x}) [\Phi_{pre}, \Phi_{post}, \Phi_{future}] \in \mathcal{E} \quad [\text{FV-Call}] \quad \Phi <: \Phi_{pre} \quad \{\Phi \circ \Phi_{post}\} e \{\Phi_e\} \quad \Phi_e <: \Phi_{future}}{\{\Phi\} f(\bar{x}); e \{\Phi_{post} \circ \Phi_e\}}$
--	---

Three main limitations:

- ✓ Inefficient entailment checking **Embed FCs into the states + Trace subtraction**
- ✓ Handle loops via unrolling **Predicates for bags of traces and FCs**
- ✓ Bug-finding (no incorrectly flagged safe code) over soundness (no missed violations)

Coq formalization

Experimental Results

Category	Example APIs	Future Conditions
1. File Ops	fopen, open fclose, close	Finally to close the file descriptor Globally do not access the file descriptor Read-only files cannot be written to
2. Threads	pthread_create pthread_mutex_lock	Finally to pthread_join or detach the thread Finally to pthread_mutex_unlock
3. Memory	free malloc realloc	Globally do not access the pointer Finally free the new pointer Globally the old pointer is not accessed & finally free the new pointer
4. Sockets	socket	Finally to close the socket
5. Database	sqlite3_open	Finally to sqlite3_close the connection
6. URV/NPD	fgets, gethostbyaddr	Check the return value immediately after calls

Write these future conditions manually

Experimental Results

Category	LoC	PrimS	InferredS	InferredInv	Report/Exp.	Time(s)
1	656	8	29	7	14/12	10.05
2	330	4	25	1	4/4	1.97
3	424	6	30	11	25/23	8.87
4	103	2	6	1	3/3	1.76
5	108	4	6	0	4/4	1.97
6	67	10	5	0	5/5	0.56
Total	1,688	34	101	20	55/51	25.18

Category	Example APIs	Future Conditions
1. File Ops	fopen, open fclose, close	Finally to close the file descriptor Globally do not access the file descriptor Read-only files cannot be written to
2. Threads	pthread_create pthread_mutex_lock	Finally to pthread_join or detach the thread Finally to pthread_mutex_unlock
3. Memory	free malloc realloc	Globally do not access the pointer Finally free the new pointer Globally the old pointer is not accessed & finally free the new pointer
4. Sockets	socket	Finally to close the socket
5. Database	sqlite3_open	Finally to sqlite3_close the connection
6. URV/NPD	fgets, gethostbyaddr	Check the return value immediately after calls

False positive due to the limited expressiveness:

```

1 void false_positive1() {
2     int** ptr1= malloc(4);
3     int*   ptr2= malloc(4);
4     *ptr1 = ptr2;
5     free(*ptr1);
6     free(ptr1); }
7 False positive: Memory Leak!

```


Future Conditions

*Thanks for
listening!*

Bug Finding and Repair

- ✓ A novel *future-condition*
- ✓ Compositional temporal analysis
- ✓ Light-weight specification inference
- ✓ Fast and most-automated
- ✓ Proof guided repair
- ✓ Large-scale usability

Verification

- ✓ Handle loops via recursive predicates
- ✓ Efficient (linear) entailment checking
- ✓ Sound weakening when path explosion
- ✓ No false negatives

- ☐ No machine checkable certification
- ☐ Limited expressiveness